

CS 423

Operating System Design:

Swapping

Sep 24

Ram Alagappan

Slide ack: Prof. Shivaram Venkataraman (Wisconsin)

Logistics

MPI is due in less than a week...take help from TAs as needed
Releasing MP2 earlier than planned (not sure).

9/29 (SOSP travel for me) – Jiyu Hu, expert in memory systems will teach this lecture

10/1 MP2 walkthrough...

AGENDA / LEARNING OUTCOMES

Recap discussion on better page tables

Support large virtual mem on small physical mem

What are the mechanisms and policies for this?

RECAP

MANY INVALID PTES

PFN	valid	prot
10	1	r-x
-	0	-
23	1	rw-
-	0	-
-	0	-
-	0	-
-	0	-
...many more invalid...		
-	0	-
-	0	-
-	0	-
-	0	-
28	1	rw-
4	1	rw-

how to avoid
storing these?

Problem: linear PT must still allocate PTE for
each page (even unallocated ones)

APPROACHES

1. Segmented Paging
2. Multi-level page tables
 - Page the page tables
 - Page the page tables of page tables...
3. Inverted page tables

Multilevel Page Table – Key Idea

Linear Page Table

PTBR 201

valid	prot	PFN	
1	rx	12	PFN 201
1	rx	13	
0	-	-	
1	rw	100	
0	-	-	PFN 202
0	-	-	
0	-	-	
0	-	-	
0	-	-	PFN 203
0	-	-	
0	-	-	
0	-	-	
0	-	-	PFN 204
0	-	-	
1	rw	86	
1	rw	15	

Multi-level Page Table

PDBR 200

	valid	PFN
PFN 200	1	201
	0	-
	0	-
	1	204

The Page Directory

PDEs

of valid bit in PDE

PTEs

	valid	prot	PFN	
PFN 201	1	rx	12	
	1	rx	13	
	0	-	-	
	1	rw	100	

[Page 1 of PT: Not Allocated]

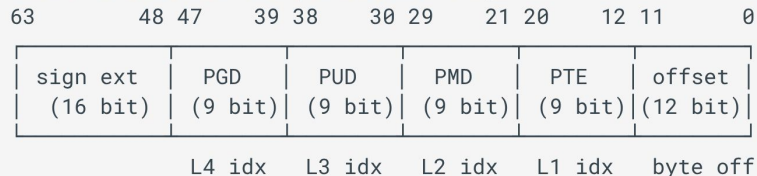
[Page 2 of PT: Not Allocated]

	valid	prot	PFN	
PFN 204	0	-	-	
	0	-	-	
	1	rw	86	
	1	rw	15	

Meaning of valid bit in PDE and PTE

Linux x86 Page Table

48-bit virtual address (canonical form):



Terminology used in the kernel source:

PGD = Page Global Directory (level 4, pointed to by CR3)

PUD = Page Upper Directory (level 3)

PMD = Page Middle Directory (level 2)

PTE = Page Table Entry (level 1, maps a 4KB page)

CR3 (physical addr of PGD)

→ PGD[VA[47:39]] (physical addr of PUD page)

→ PUD[VA[38:30]] (physical addr of PMD page)

→ PMD[VA[29:21]] (physical addr of PTE page)

→ PTE[VA[20:12]] (physical addr of 4KB page frame)

+ VA[11:0] (byte offset within the page)

= Physical Address

Remember: TLB caches translations for these tables as well (not just last level)

5-level paging: modern x86 can extend 48-bit VA to 57-bit VA by adding another level.

INVERTED PAGE TABLE

A single system-wide table mapping PFN to VPN + ASID

Naïve approach:

- Search through data structure $\langle \text{ppn}, \text{vpn} + \text{asid} \rangle$ to find match

- Too much time to search entire table

Better:

- Maintain a hash-map of $\text{vpn} + \text{asid}$ to ppn

- Smaller number of entries to search for exact match

Used in IBM PowerPC

- Typically requires TLB miss handling by software (OS)

END RECAP

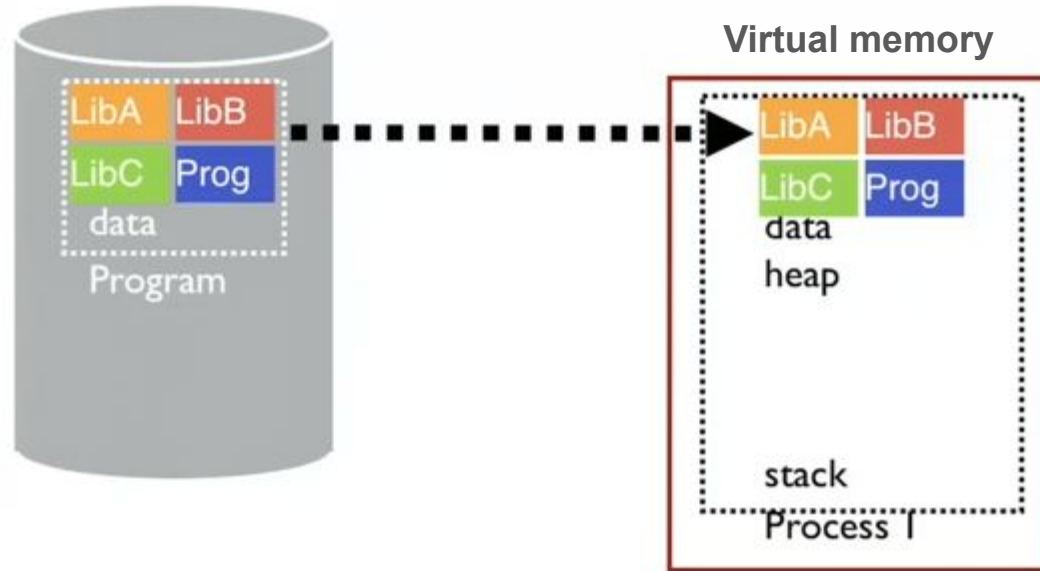
SWAPPING

ONE LAST PROBLEM

Memory virtualization thus far:

- Support multiple processes, each with its virtual memory
- The virtual memory for a process is contiguous
- ...but it's physical memory doesn't need to be
- Solved external fragmentation using paging
- The page table doesn't need to be contiguous - page it!
- Use hardware support for faster translation: TLB

One remaining problem: **can't fit everything in physical memory**

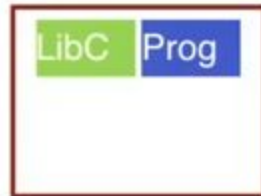


Code: many large libraries, some of which are rarely/never used

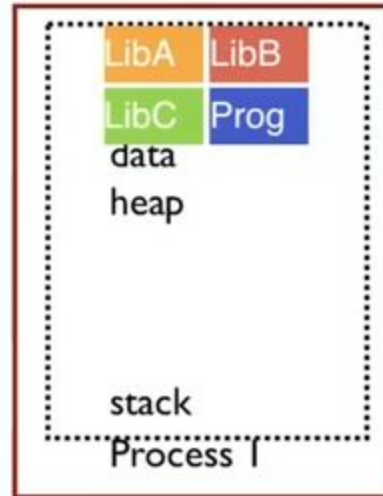
How to avoid wasting physical pages to store rarely used virtual pages?

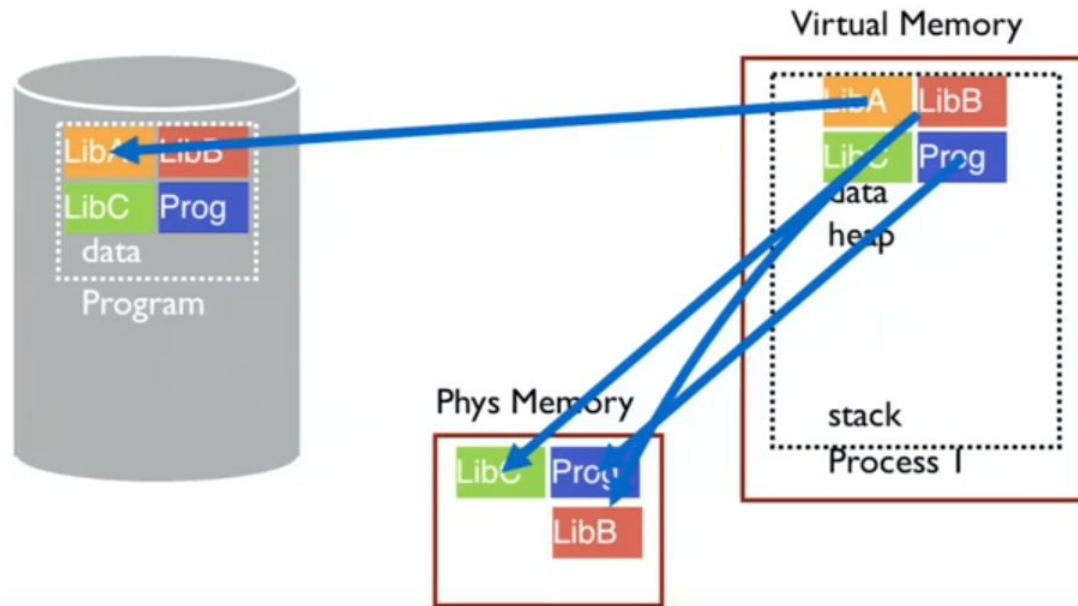


Phys Memory



Virtual Memory





copy (or move) to RAM

Called "**paging**" in

Locality of Reference

locality of reference within processes

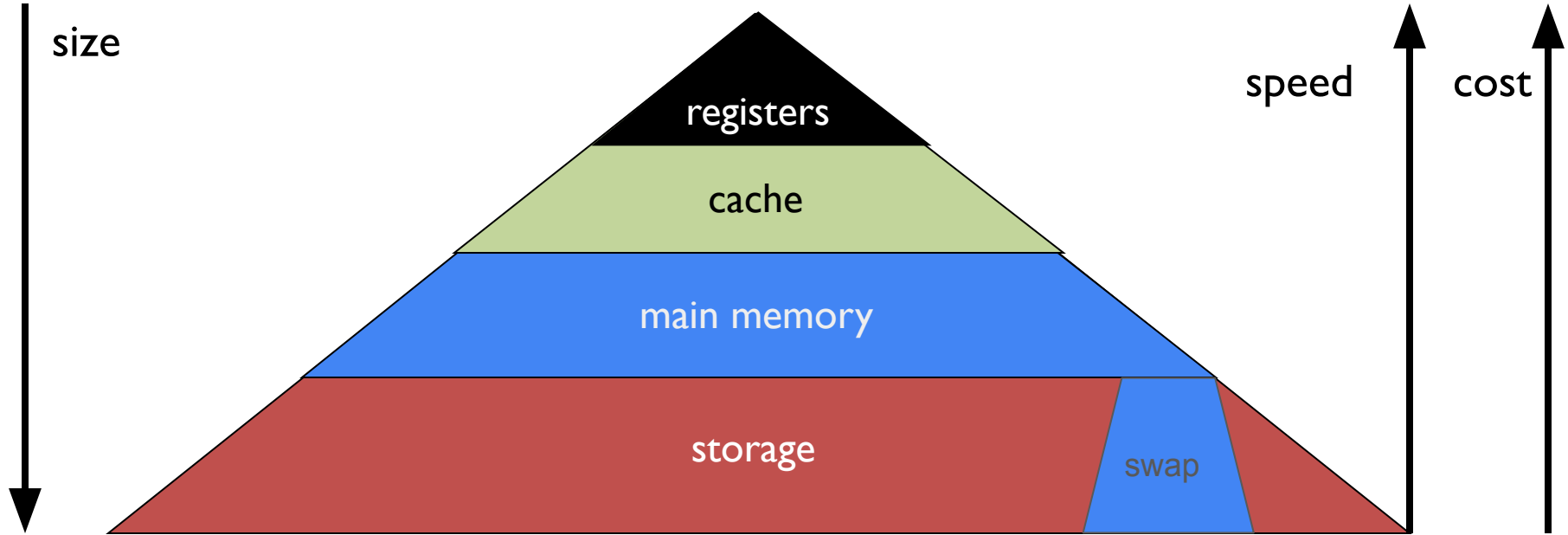
- **Spatial:** reference memory addresses **near** previously referenced addresses
- **Temporal:** reference memory addresses that have referenced in the past
- Processes spend majority of time in small portion of code
 - Estimate: 90% of time in 10% of code

Implication:

- Process only uses small amount of address space at any moment
- Only small amount of address space must be resident in physical memory

Memory Hierarchy

Leverage **memory hierarchy** of machine architecture
Each layer acts as “backing store” for layer above



Swap Space

Designated & reserved fraction of persistent storage

- Sizing is configurable

Persistent storage is accessed in units of “blocks”

- Typically, block size is equal to or a multiple of page size.
- “Paging out”: write a page out to swap space
- “Paging in”: read a page in from swap space

OS tracks free space in the swap space

- Simple block addressing: linearly from 0 to n

SWAPPING Intuition

OS keeps unreferenced/unneeded pages in swap space

- Slower, cheaper storage backing the memory

Process can run even when all its pages are not in main memory

OS and h/w cooperate to make storage seem like memory

- Illusion: process address space is entirely in main memory

Requirements:

- **Mechanism:** locate (move) pages in (between) memory and storage
- **Policy:** determine which pages to move, and when

Note: Books/Internet refers to swap space as disk (can be SSD)

SWAPPING

Question 1: Can a USB-stick serve as swap space? Problem?

Question 2: What happens to swap space when the OS is restarted, i.e., machine is rebooted?

Question 3: What happens in swap space when a laptop hibernates?

Question 3: What if (total memory to all processes) > (RAM + swap space)?

Virtual Address Space Mechanisms

Each page in virtual address space maps to one of three locations:

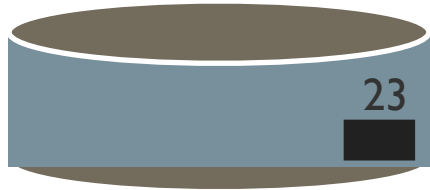
- Physical main memory: fast
- Swap Space (backing store): slow
- Nothing: Free

Extend page tables with an extra bit: present

- permissions (r/w), valid, dirty, present
- Page in memory: present bit set in PTE
- Page on disk: present bit cleared
 - PTE points to block on disk (use the same bits for PFN or block#)
 - Causes trap into OS when page is not in memory: **Page Fault**
 - OS reads page from swap space and puts it into memory

When vpn 0x2 is accessed

Disk



Phys Memory



PFN	valid	prot	present
10	1	r-x	1
-	0	-	-
23	1	rw-	0
-	0	-	-
-	0	-	-
-	0	-	-
-	0	-	-
-	0	-	-
-	0	-	-
-	0	-	-
28	1	rw-	0
4	1	rw-	1

BITS: valid, present, dirty

Valid:

- Virtual memory address has been allocated
- So was mapped to physical memory at some point in the past

Present:

- The page currently resides in physical memory

Dirty:

- Page has been written to by the process
- If not dirty, can avoid writing to swap when being evicted

x86: no separate valid bit, only present bit

Present bit 0 means HW cannot use the PTE for translation

When program tries to access a page and HW finds from PTE that present = 0, it simply raises exception to the OS...

Can mean two different things

1. Invalid mapping
2. Page not present in physical mem

How does Linux know this is a valid mapping but page is on swap vs. this is invalid mapping?

63	52 51		12 11	0
+-----+-----+-----+-----+				
NX	misc/reserved		Physical Page Number	flags
+-----+-----+-----+-----+				
Typical low-order flags:				
bit 8	G	Global		
bit 7	PAT			
bit 6	D	Dirty		
bit 5	A	Accessed		
bit 4	PCD	Cache disable		
bit 3	PWT	Write-through		
bit 2	U/S	User / Supervisor		
bit 1	R/W	Read / Write		
bit 0	P	Present		

VMA (virtual memory areas)

Linux's **VMA metadata** is the kernel's per-process record of which virtual-address ranges are valid and what properties each range has, such as permissions, backing file/anonymous memory, and sharing behavior.

OS asks “Does this VA fall inside any valid VMA, and is this kind of access allowed there?”

If no, then segfault.

otherwise, use the bits in the PTE to find
the swap address to page in

code	0x400000 - 0x420000	r-x
data	0x420000 - 0x430000	rw-
heap	0x600000 - 0x680000	rw-
mmap()	...	
stack	0x7fff...	rw-

Virtual Memory: Full Mechanism

First, hardware checks TLB for virtual address

- if TLB hit, address translation is done; page in physical memory

Else **//TLB miss**

- Hardware or OS walk page tables
- If PTE valid + present bits set, then page in physical memory

- Insert PTE into TLB, retry instruction

Else (valid and/or present is 0) **//Page fault**

- Trap into OS (not handled by hardware)

- Find free PFN. If necessary,

- Select victim page in memory to kick out

- If modified (dirty bit set), page out victim page to swap

- Kick off I/O to read the page from storage to PFN

- Process transitions to BLOCKED, OS does a context switch

Virtual Memory: Full Mechanism (continued)

The read I/O was tagged with PID, PTE, destination PFN, etc.

When read I/O completes, interrupt handler runs

- Updates PTE with new PFN
- Sets the present bit
- Makes the original process (PID) runnable/ready

When process runs:

- Wakes up in kernel mode in the page fault handler
- Returns to user mode to retry instruction
- Results in TLB miss
 - Will find PTE with present bit
 - // insert into TLB and retry instruction

Why not insert PTE into TLB at the end of the page-fault interrupt handler?

Interaction With OS Scheduler

During TLB miss handling, process is in _____ state

(Hard) page faults are expensive: OS must read from disk

During page fault handling, process is in _____ state

Once page fault handling kicks off the read I/O, process moves to _____ state

The process moves to _____ state once the read I/O completes and sets the PTE's present bit.

Soft vs. Hard Page Faults

Hard: Expensive (process transitions to BLOCKED)

Requires reading the page from disk

Soft: Cheap (no context switch required)

Page already in memory, but OS need to do some work,

E.g., COW when a fork(ed) child dirties a page

Both parent and child have $w = 0$ in PTE, when write happens, HW raises exception, OS copies the page

But, how does OS know whether this is a real permission error vs. copy on write?

SWAP SPACE LOCATION

The fact that swap space is persistent is somewhat *incidental*

It happens to be the next tier in the memory hierarchy!

Persistence allows features, such as laptop hibernation

Smartphones typically use zRAM instead

Local: SSD or hard drive or USB device

Remote: on a different machine!

Remote Swapping

RPC

RDMA!

RDMA for remote memory swapping

REPLACEMENT

Does not really occur 1 page at a time

Inefficient to wait until memory is entirely full!

Swap/Page daemon: background process

Low & High watermarks (LW & HW)

When $\#free\text{-}pages < LW$

Evict sufficient pages until $\#free\text{-}page > HW$

Go to sleep

Advantages?

Virtual Memory: Full Mechanism 2 (modification)

First, hardware checks TLB for virtual address

- if TLB hit, address translation is done; page in physical memory

Else **//TLB miss**

- Hardware or OS walk page tables
- If PTE present bit set, then page in physical memory
 - Insert PTE into TLB, retry instruction

Else **//Page fault**

- Trap into OS (not handled by hardware)
- Find free PFN. ~~If necessary,~~ ← if #free-pages < LW
 - ~~Select victim page in memory to kick out~~ ← wake up daemon
 - ~~If modified (dirty bit set), page out victim page to swap~~
 - ← if no free-pages, sleep for daemon
- Kick off I/O to read the page from swap space to PFN
- Process is marked as BLOCKED, OS does a context switch

SWAPPING POLICIES

SWAPPING Policies

Goal: Minimize number of page faults

- Page faults require millisecs to handle (reading from swap)
- Implication: Lots of time for OS to make good decisions

OS has two decisions

- **Page selection**
When should a page (or pages) on disk be **brought into** memory?
- **Page replacement**
Which resident page (or pages) in memory should be **kicked out**?

Page Selection

Demand paging: Load page only when page fault occurs

- Intuition: Wait until the last minute; when page is absolutely needed
- When a process starts up: No pages are loaded in memory
- Problem: Pay cost of page fault for every newly accessed page

Pre-paging (anticipatory, prefetching): Load page beforehand

- OS predicts future access and prefetches pages into memory
- Works well for some access patterns (e.g., sequential)
- Two costs of bad prefetching: Wasted I/O & kicking out page that might be needed

Some combination of the two, together with hints

Hints: User-supplied hints about page references

Hints with `madvise()`

`int madvise(void *addr, size_t length, int advice)` - allows user to give hints to OS

`MADV_RANDOM` // kernel may disable prefetching

`MADV_SEQUENTIAL` // kernel may aggressively prefetch...

 // ...and kick out after access

`MADV_WILLNEED`

`MADV_DONTNEED`

and more...

Generally, hard to tune and extract better performance than OS default

Page Replacement

One potential dimension for selecting victim pages to kick out:

- Write out victim page to disk if modified (dirty bit set)
- If victim page is not modified (clean), just discard
- So when it makes sense, **pick clean pages as victims**

Now the policies

OPTIMAL: Replace the page to be accessed furthest in the future

- Pro: Guaranteed to minimize #page faults
- Con: Requires knowing the future! Impractical, but good for comparisons

Page Replacement

FIFO: Replace page that has been in memory the longest

- Intuition: Referenced long time ago, get rid of it
- Advantage: Fair as each page gets equal residency; Easy to implement
- Disadvantage: Some pages may remain popular

LRU: Least-recently-used

- Intuition: Recent past predicts the near future
- Advantages: With locality, LRU approximates OPTIMAL
- Disadvantages:
 - Harder to implement, must track which pages have been accessed

Page Replacement - chat for 2 mins

Page reference string: ABCABDADBCB

Metric:
Miss count

Three pages
of physical
memory

	OPT	FIFO	LRU									
ABC	<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>			
A	<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>			
B	<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>			
D	<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>			
A	<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>			
D	<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>			
B	<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>			
C	<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>			
B	<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>			

Page Replacement

Page reference string: ABCABDADBCB

Metric:
Miss count

Three pages
of physical
memory

	OPT	FIFO	LRU									
ABC	<table><tr><td>A</td><td>B</td><td>C</td></tr></table>	A	B	C	<table><tr><td>A</td><td>B</td><td>C</td></tr></table>	A	B	C	<table><tr><td>A</td><td>B</td><td>C</td></tr></table>	A	B	C
A	B	C										
A	B	C										
A	B	C										
A	<table><tr><td>A</td><td>B</td><td>C</td></tr></table>	A	B	C	<table><tr><td>A</td><td>B</td><td>C</td></tr></table>	A	B	C	<table><tr><td>A</td><td>B</td><td>C</td></tr></table>	A	B	C
A	B	C										
A	B	C										
A	B	C										
B	<table><tr><td>A</td><td>B</td><td>C</td></tr></table>	A	B	C	<table><tr><td>A</td><td>B</td><td>C</td></tr></table>	A	B	C	<table><tr><td>A</td><td>B</td><td>C</td></tr></table>	A	B	C
A	B	C										
A	B	C										
A	B	C										
D	<table><tr><td>A</td><td>B</td><td>D</td></tr></table>	A	B	D	<table><tr><td>D</td><td>B</td><td>C</td></tr></table>	D	B	C	<table><tr><td>A</td><td>B</td><td>D</td></tr></table>	A	B	D
A	B	D										
D	B	C										
A	B	D										
A	<table><tr><td>A</td><td>B</td><td>D</td></tr></table>	A	B	D	<table><tr><td>D</td><td>A</td><td>C</td></tr></table>	D	A	C	<table><tr><td>A</td><td>B</td><td>D</td></tr></table>	A	B	D
A	B	D										
D	A	C										
A	B	D										
D	<table><tr><td>A</td><td>B</td><td>D</td></tr></table>	A	B	D	<table><tr><td>D</td><td>A</td><td>C</td></tr></table>	D	A	C	<table><tr><td>A</td><td>B</td><td>D</td></tr></table>	A	B	D
A	B	D										
D	A	C										
A	B	D										
B	<table><tr><td>A</td><td>B</td><td>D</td></tr></table>	A	B	D	<table><tr><td>D</td><td>A</td><td>B</td></tr></table>	D	A	B	<table><tr><td>A</td><td>B</td><td>D</td></tr></table>	A	B	D
A	B	D										
D	A	B										
A	B	D										
C	<table><tr><td>C</td><td>B</td><td>D</td></tr></table>	C	B	D	<table><tr><td>C</td><td>A</td><td>B</td></tr></table>	C	A	B	<table><tr><td>C</td><td>B</td><td>D</td></tr></table>	C	B	D
C	B	D										
C	A	B										
C	B	D										
B	<table><tr><td>C</td><td>B</td><td>D</td></tr></table>	C	B	D	<table><tr><td>C</td><td>A</td><td>B</td></tr></table>	C	A	B	<table><tr><td>C</td><td>B</td><td>D</td></tr></table>	C	B	D
C	B	D										
C	A	B										
C	B	D										

LRU vs. OPT

QUESTION: Think of a workload & config where LRU will be way worse than OPT. Random replacement might do better than LRU!

Page Replacement Comparison

What happens to performance with increase in physical memory size?

LRU, OPT:

- Guaranteed to have fewer (or same number of) page faults
- Stack property:
 - cache of size $N+1$ includes contents of cache of size N

FIFO:

- Usually have fewer page faults
- Belady's anomaly: May actually have **more** page faults!

FIFO PERFORMANCE MAY DECREASE!

Consider access stream: ABCDABEABCDE

Physical memory size: 3 pages vs. 4 pages

How many misses with FIFO?

9 misses with 3 pages and 10 misses with 4 pages

3 Pages

A, B, C, D, A, B, E miss

A hit, B hit, C, D miss

E hit

4 pages

A, B, C, D miss

A hit, B hit, E miss –

A, B, C, D, E miss

IMPLEMENTING LRU

Software Perfect LRU

- OS maintains ordering of physical pages by reference time
- When page is referenced: modify the ordering
- When need victim: Pick page with oldest time
- Trade-off: Slow on memory reference, fast on replacement

Hardware Perfect LRU

- Associate timestamp with each page
- When page is referenced: Store system clock with page
- When need victim: Scan to find oldest timestamp
- Trade-off: Fast on memory reference, slow on replacement (especially as size of memory grows)

In practice, do not implement Perfect LRU

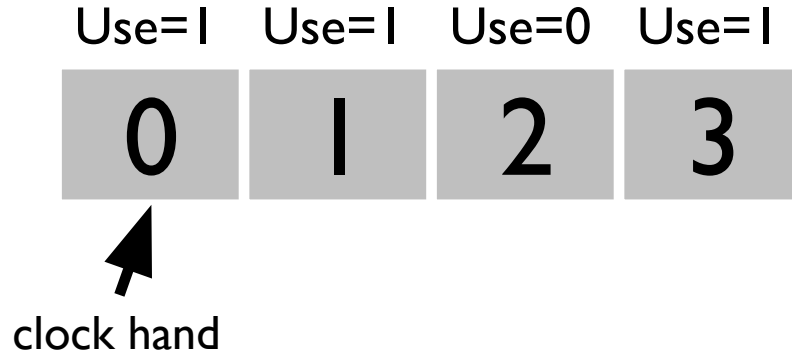
- LRU is an approximation anyway, so approximate some more :-)
- Goal: Find an old page, but not necessarily the very oldest

CLOCK ALGORITHM: APPROX LRU

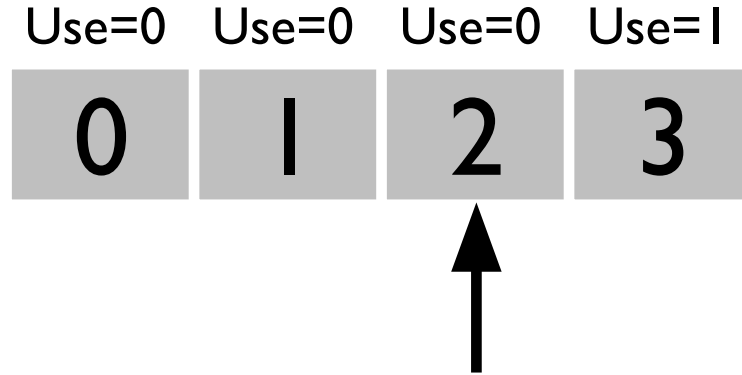
- PTE comprises a use/reference bit
- Is set whenever a page is referenced
- Page replacement: Look for page with use bit cleared (has not been referenced for awhile)
- Implementation:
 - Keep pointer to last examined page frame
 - Traverse pages in circular fashion
 - Clear use bits as search
 - Stop when find page with already cleared use bit, replace this page

CLOCK: LOOK FOR A PAGE

Physical Mem:



Evict
contents of
frame 2, load
in new page



CLOCK EXTENSIONS

Replace multiple pages at once

- Intuition: Efficient to write pages to swap space in bulk
- Find multiple victims each time and track free list

Use dirty bit to bias against replacing dirty pages

- Intuition: More expensive to replace dirty pages
Dirty pages must be written to disk, clean pages do not
- Replace pages that have use bit and dirty bit cleared

Add software counter (“unused”)

Intuition: Can tell how often pages are accessed

Increment software counter if use bit is 0

Replace when change exceeds some specified limit

LINUX 2Q

Linux has a unified page cache (will cover later in course)

Maintains active list vs inactive list

On first reference, put page in inactive list

- If accessed again, promoted to active list

- Pick victim pages from inactive list in FIFO order to swap out

In background: move pages from active to inactive list until active list is $\leq$ 2/3rd of physical memory size

- Protects against larger-than-cache-cyclic-access pattern (will cover later)

LINUX CODE

`include/linux/swap.h`

Data structure used to track swap area

`struct swap_info_struct`

Operates in units of 256 pages “clusters”

`struct swap_cluster_info`

Parallelism: each CPU is given a cluster

OUT OF MEMORY

What happens when total memory allocated to all processes > RAM + swap space?

1. OS constantly pages in/out from swap space—aka *thrashing*

ProcessA kicks out p2 to page in p1, ProcessB kicks out p1 to page in p2, and so on...

System noticeably slows down

2. OOM Killer (kernel) gets invoked

Kill processes that hog memory; typically younger processes

3. User processes: malloc() returns null

SUMMARY: VIRTUAL MEMORY

Abstraction: Virtual address space with code, heap, stack

Address translation

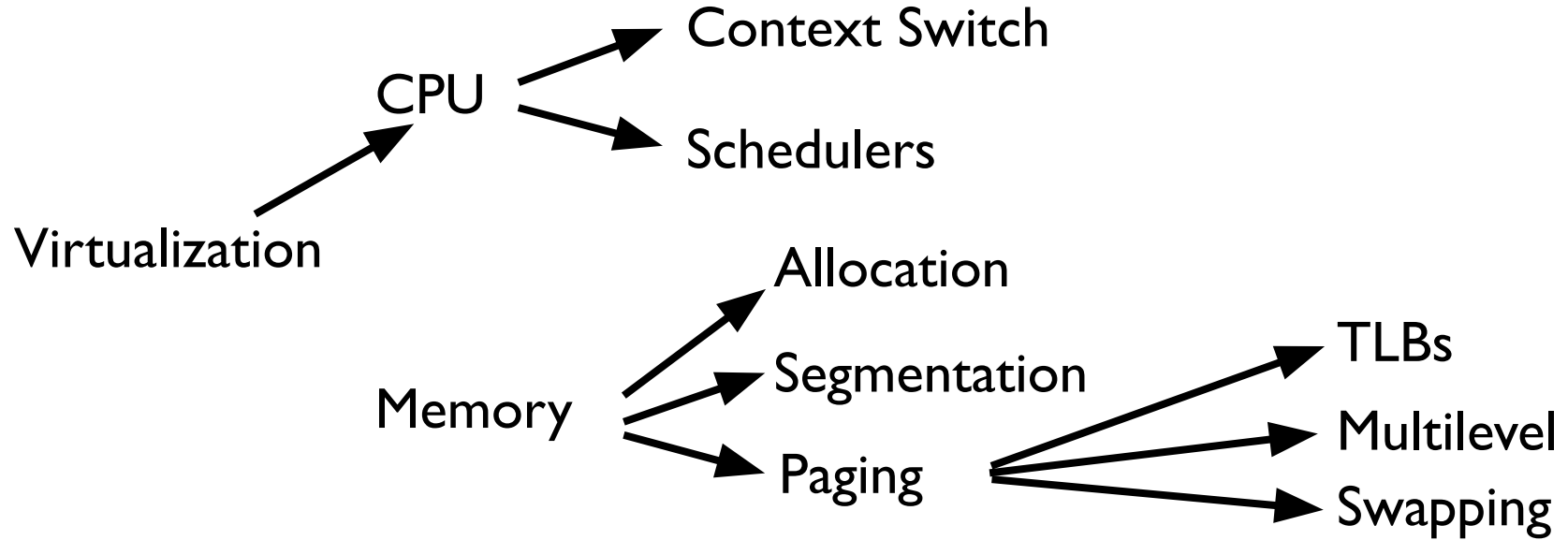
- Contiguous memory: base, bounds, segmentation
- Using fixed sizes pages with page tables

Challenges with paging

- Extra memory references: avoid with TLB
- Page table size: avoid with multi-level paging, inverted page tables etc.

Larger address spaces: Swapping mechanisms, policies (LRU, Clock)

REVIEW: EASY PIECE 1



NEXT CLASS

Concurrency!